ДЕПОНИРУЕМЫЕ МАТЕРИАЛЫ, ИДЕНТИФИЦИРУЮЩИЕ ПРОГРАММУ ДЛЯ ЭВМ

«ПВ-Система PDF»

Версия 3.55.0

Правообладатель:
Ипатов Сергей Генадьевич

Автор(ы):
Ипатов Сергей Генадьевич

Вид материалов:
Фрагменты исходного текста: страницы 1–30 и 1020–1049 из 1049 страниц полного текста. В скобках внизу каждой страницы указан её номер в полном тексте.

Языки программирования:
C#, JavaScript (TypeScript), Python

Объём исходного текста:
1 265 Кбайт (1 295 131 байт)

Сибай, 2026

```
// ===== desktop/csharp/PvsPdfApp/Program.cs =====
using System.IO.Pipes;
using System.Text;

namespace PvsPdfApp;

internal static class Program
{
    // Папка программы на диске C
    public const string InstallDir = @"C:\PVSPDF";

    public const string PipeName = "PVSPDF_Open_Pipe";

    [STAThread]
    static void Main(string[] args)
    {
        // Пользователь мог выделить сразу несколько документов
  –
        // каждый откроется своей вкладкой
        string[] openFiles = args
            .Where(a => a.EndsWith(".pdf", StringComparison.Ordi
  nalIgnoreCase) && File.Exists(a))
            .ToArray();

        // Один экземпляр программы
        using var mutex = new Mutex(true, "PVSPDF_SingleInstance
  ", out bool isNew);
        if (!isNew)
        {
            // Программа уже открыта: передаём ей документы и вы
  ходим,
            // они добавятся вкладками в уже запущенном окне
            if (!SendToRunning(openFiles))
            {
                MessageBox.Show(
                    "ПВ-Система PDF уже запущена.",
                    "ПВ-Система PDF",
```

```
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Information);
            }
            return;
        }

        // Готовим просмотрщик заранее, параллельно с открытием
окна:
        // это самая долгая часть запуска, и ждать её впустую не
зачем
        MainForm.WarmUp();

        // Старые журналы убираем в фоне — запуск не ждёт
        _ = Task.Run(AppLog.Cleanup);

        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.SetHighDpiMode(HighDpiMode.PerMonitorV2);

        Application.ThreadException += (s, e) => ShowFatal(e.Exc
eption);
        AppDomain.CurrentDomain.UnhandledException += (s, e) =>
ShowFatal(e.ExceptionObject as Exception);

        Application.Run(new MainForm(openFiles));
    }

    static bool SendToRunning(string[] files)
    {
        try
        {
            using var pipe = new NamedPipeClientStream(".", Pipe
Name, PipeDirection.Out);
            pipe.Connect(2000);
            // Документы разделяем переводом строки: в именах фа
йлов
            // такого символа быть не может
```

```
            byte[] data = Encoding.UTF8.GetBytes(string.Join("\n
  ", files));
            pipe.Write(data, 0, data.Length);
            pipe.Flush();
            return true;
        }
        catch
        {
            return false;
        }
    }

    static void ShowFatal(Exception? ex)
    {
        AppLog.Error("сбой", "Необработанная ошибка программы",
  ex);

        MessageBox.Show(
            "Произошла ошибка:\r\n\r\n" + (ex?.Message ?? "неизв
  естная ошибка") +
            "\r\n\r\nПодробности записаны в журнал: " + AppLog.D
  ir,
            "ПВ-Система PDF",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

```
// ===== desktop/csharp/PvsPdfApp/MainForm.cs =====
using System.Text.Json;
using Microsoft.Web.WebView2.Core;
using Microsoft.Web.WebView2.WinForms;

namespace PvsPdfApp;

public class MainForm : Form
{
    readonly WebView2 _web = new();

    // Подготовка встроенного просмотрщика — самая долгая часть
  запуска,
    // поэтому начинаем её заранее и не ждём завершения.
    //
    // ВАЖНО: только из главного потока программы. Просмотрщик з
  аводится
    // на том же потоке, где потом работает, и запуск из фоновог
  о
    // заканчивался отказом «изменение режима для потока невозмо
  жно»
    static Task<CoreWebView2Environment>? _envReady;

    public static void WarmUp()
    {
        if (_envReady != null) return;
        try
        {
            string dataDir = Path.Combine(Program.InstallDir, "d
  ata");
            Directory.CreateDirectory(dataDir);

            var opts = new CoreWebView2EnvironmentOptions
            {
                // Отключаем только то, чем программа заведомо н
  е пользуется:
                // предложение перевести страницу, передачу виде
```

```
o на телевизор
                // и проверку «браузер по умолчанию».
                //
                // Намеренно НЕ трогаем встроенный просмотрщик P
DF (msPdfOOUI):
                // на нём держится печать документов
                AdditionalBrowserArguments =
                    "--disable-features=Translate,MediaRouter "
+
                    "--no-default-browser-check --no-first-run",
            };

            // Работа уходит вперёд сама, ждать её здесь не нужн
o —
            // окно тем временем продолжает открываться
            _envReady = CoreWebView2Environment.CreateAsync(null
, dataDir, opts);
        }
        catch
        {
            // Не вышло начать заранее — не беда, окно подготови
т
            // просмотрщик обычным порядком
            _envReady = null;
        }
    }

    readonly string[] _openFiles;
    readonly DateTime _startedAt = DateTime.UtcNow;
    string _webRoot = "";
    string _openDir = "";
    CancellationTokenSource? _scanCancel;

    // Есть ли в открытых документах правки, которых нет в файла
х.
    // Об этом сообщает сам интерфейс при каждом изменении
    bool _unsaved;
```

```
  string _unsavedNames = "";
  // Человек уже ответил на вопрос о несохранённом — закрываем
ся молча
  bool _closeConfirmed;

  public MainForm(string[]? openFiles = null)
  {
      _openFiles = openFiles ?? Array.Empty<string>();

      Text = "ПВ-Система PDF";
      Width = 1280;
      Height = 860;
      MinimumSize = new Size(900, 600);
      StartPosition = FormStartPosition.CenterScreen;
      BackColor = ColorTranslator.FromHtml("#F7F6F2");

      try
      {
          string ico = Path.Combine(AppContext.BaseDirectory,
"pvspdf.ico");
          if (File.Exists(ico)) Icon = new Icon(ico);
      }
      catch { }

      _web.Dock = DockStyle.Fill;
      _web.DefaultBackgroundColor = ColorTranslator.FromHtml("
#F7F6F2");
      Controls.Add(_web);

      Load += async (s, e) => await StartAsync();
  }

  async Task StartAsync()
  {
      _webRoot = Path.Combine(AppContext.BaseDirectory, "web")
;
      if (!File.Exists(Path.Combine(_webRoot, "index.html")))
```

```
        {
            MessageBox.Show(
                "Не найдены файлы интерфейса программы (папка we
b).\r\n" +
                "Переустановите ПВ-Система PDF.",
                "ПВ-Система PDF", MessageBoxButtons.OK, MessageB
oxIcon.Error);
            Close();
            return;
        }

        // Подготовка началась ещё до появления окна — здесь обы
чно
        // остаётся только забрать готовый результат
        try
        {
            WarmUp();
            var env = await _envReady!;
            await _web.EnsureCoreWebView2Async(env);
        }
        catch
        {
            // Заранее не получилось — заводим просмотрщик обычн
ым
            // порядком. Так программа откроется даже там, где р
анняя
            // подготовка почему-то не сработала
            _envReady = null;
            try
            {
                string dataDir = Path.Combine(Program.InstallDir
, "data");
                Directory.CreateDirectory(dataDir);
                var env = await CoreWebView2Environment.CreateAs
ync(null, dataDir);
                await _web.EnsureCoreWebView2Async(env);
            }
```

```
            catch (Exception ex)
            {
                AppLog.Error("запуск", "Не удалось запустить про
смотрщик WebView2", ex);
                ShowViewerError(ex);
                return;
            }
        }

        var core = _web.CoreWebView2;

        core.Settings.AreDefaultContextMenusEnabled = false;
        core.Settings.AreDevToolsEnabled = false;
        core.Settings.IsStatusBarEnabled = false;
        core.Settings.AreBrowserAcceleratorKeysEnabled = false;
        core.Settings.IsSwipeNavigationEnabled = false;

        // Локальные файлы интерфейса отдаются как https://pvspd
f.local/
        core.SetVirtualHostNameToFolderMapping(
            "pvspdf.local", _webRoot, CoreWebView2HostResourceAc
cessKind.Allow);

        // Открываемые документы отдаются как https://pvspdf.fil
e/ — программа
        // читает их напрямую, без перекодирования: большой файл
 открывается
        // в разы быстрее и не занимает лишнюю память
        _openDir = Path.Combine(Program.InstallDir, "open");
        Directory.CreateDirectory(_openDir);
        core.SetVirtualHostNameToFolderMapping(
            "pvspdf.file", _openDir, CoreWebView2HostResourceAcc
essKind.Allow);

        // Прошлые документы удаляем в фоне: на медленном диске
перебор папки
        // задерживал появление окна
```

```
        _ = Task.Run(CleanOpenDir);

        // Открывать внешние ссылки в системном браузере
        core.NewWindowRequested += (s, e) =>
        {
            e.Handled = true;
            try
            {
                System.Diagnostics.Process.Start(new System.Diag
nostics.ProcessStartInfo
                {
                    FileName = e.Uri,
                    UseShellExecute = true
                });
            }
            catch { }
        };

        core.WebMessageReceived += OnWebMessage;

        // Сообщаем странице, что она работает в десктопной верс
ии.
        // Список принтеров сюда не кладём: опрос сетевых принте
ров
        // занимает секунды и задерживал бы запуск программы
        await core.AddScriptToExecuteOnDocumentCreatedAsync(
            "window.PVSPDF_DESKTOP = true;" +
            "window.PVSPDF_VERSION = '" + AppVersion() + "';" +
            "window.PVSPDF_PRINTER = " + JsonSerializer.Serializ
e(Settings.Printer ?? "") + ";" +
            "window.PVSPDF_PRINTERS = [];" +
            // Решение о полной версии принимает программа, а не
 страница:
            // подделать его правкой памяти браузера нельзя
            "window.PVSPDF_FULL = " + (License.IsFull() ? "true"
 : "false") + ";" +
            "window.PVSPDF_LIC = " + JsonSerializer.Serialize(ne
```

```
w
            {
                org = License.Org,
                key = License.Key,
                until = License.Until,
                machine = License.MachineId(),
                machineName = License.MachineName(),
            }) + ";");

        core.Navigate("https://pvspdf.local/index.html");

        core.NavigationCompleted += async (s, e) =>
        {
            if (_openFiles.Length > 0)
            {
                // Небольшая пауза: интерфейс должен успеть подг
отовиться
                // к приёму документов
                await Task.Delay(150);
                foreach (string f in _openFiles) await SendFileA
sync(f);
            }
            SendPrintersAsync();
        };

        StartPipeListener();
    }

    // Принимаем файлы от повторных запусков программы:
    // документ откроется в этом окне вместо сообщения "уже запу
щена"
    void StartPipeListener()
    {
        _ = Task.Run(async () =>
        {
            while (!IsDisposed)
            {
```

```
            try
            {
                using var pipe = new System.IO.Pipes.NamedPi
peServerStream(
                    Program.PipeName, System.IO.Pipes.PipeDi
rection.In);
                await pipe.WaitForConnectionAsync();

                using var reader = new StreamReader(pipe, Sy
stem.Text.Encoding.UTF8);
                string payload = (await reader.ReadToEndAsyn
c()).Trim();
                string[] paths = payload
                    .Split('\n', StringSplitOptions.RemoveEm
ptyEntries)
                    .Select(p => p.Trim())
                    .Where(File.Exists)
                    .ToArray();

                if (IsDisposed || Disposing || !IsHandleCrea
ted) break;
                BeginInvoke(new Action(async () =>
                {
                    if (IsDisposed || Disposing) return;
                    if (WindowState == FormWindowState.Minim
ized)
                        WindowState = FormWindowState.Normal
;
                    Activate();
                    BringToFront();

                    // Каждый документ откроется своей вклад
кой
                    foreach (string p in paths) await SendFi
leAsync(p);
                }));
            }
```

```
                catch
                {
                    await Task.Delay(500);
                }
            }
        });
    }

    // Принтеры ищем в фоне, чтобы окно открывалось сразу.
    // Готовый список передаём в интерфейс отдельным сообщением
    void SendPrintersAsync()
    {
        _ = Task.Run(() =>
        {
            string json = JsonSerializer.Serialize(PrinterList()
);
            try
            {
                if (IsDisposed || Disposing || !IsHandleCreated)
 return;
                BeginInvoke(new Action(() =>
                {
                    try
                    {
                        if (IsDisposed || Disposing) return;
                        _web.CoreWebView2?.ExecuteScriptAsync(
                            "window.PVSPDF_PRINTERS = " + json +
 ";" +
                            "window.dispatchEvent(new CustomEven
t('pvspdf-printers'));");
                    }
                    catch { }
                }));
            }
            catch { }
        });
    }
```

```
  // Список принтеров, установленных в Windows.
  // Первым идёт принтер по умолчанию
  static string[] PrinterList()
  {
      try
      {
          var all = System.Drawing.Printing.PrinterSettings.In
stalledPrinters
              .Cast<string>()
              .Where(p => !string.IsNullOrWhiteSpace(p))
              .Distinct()
              .ToList();

          string def = "";
          try { def = new System.Drawing.Printing.PrinterSetti
ngs().PrinterName ?? ""; } catch { }

          return all
              .OrderByDescending(p => string.Equals(p, def, St
ringComparison.OrdinalIgnoreCase))
              .ThenBy(p => p)
              .ToArray();
      }
      catch
      {
          return Array.Empty<string>();
      }
  }

  static string AppVersion()
  {
      try
      {
          string f = Path.Combine(AppContext.BaseDirectory, "a
pp_version.txt");
          if (File.Exists(f)) return File.ReadAllText(f).Trim(
```

```
);
      }
      catch { }
      return "1.0.0";
  }

  // Просмотрщик не завёлся совсем — почти всегда это значит, что на
  // компьютере не установлен нужный компонент Windows
  void ShowViewerError(Exception ex)
  {
      MessageBox.Show(
          "Не удалось запустить встроенный просмотрщик.\r\n\r\n" +
          "Установите компонент Microsoft Edge WebView2 Runtime:\r\n" +
          "https://developer.microsoft.com/microsoft-edge/webview2/\r\n\r\n" +
          ex.Message,
          "ПВ-Система PDF", MessageBoxButtons.OK, MessageBoxIcon.Error);
      Close();
  }

  // Убираем документы, оставшиеся от прошлых запусков.
  // Файлы текущего сеанса не трогаем: их мог только что открыть пользователь
  void CleanOpenDir()
  {
      DateTime started = _startedAt;
      try
      {
          foreach (string f in Directory.GetFiles(_openDir))
          {
              try
              {
                  if (File.GetCreationTimeUtc(f) >= started) c
```

```
ontinue;
                File.Delete(f);
            }
            catch { }
        }

        // Снимки сканера лежат в отдельных папках — убираем
 те,
        // что остались от прошлых запусков, чтобы диск не з
абивался
        foreach (string d in Directory.GetDirectories(_openD
ir))
        {
            try
            {
                if (Directory.GetCreationTimeUtc(d) >= start
ed) continue;
                Directory.Delete(d, true);
            }
            catch { }
        }
    }
    catch { }
 }

 async Task SendFileAsync(string path)
 {
     try
     {
         // Документ становится доступен интерфейсу по адресу
 и читается
         // по частям: просмотрщик берёт только нужные страни
цы
         string name = Path.GetFileName(path);
         string temp = Path.Combine(_openDir, "doc_" + DateTi
me.Now.Ticks + ".pdf");
```

```
            // Если документ лежит на том же диске, делаем на не
го ссылку:
            // копирование не нужно, и файл на сотни мегабайт от
крывается сразу
            if (!NativeMethods.CreateHardLink(temp, path, IntPtr
.Zero))
            {
                using var src = new FileStream(path, FileMode.Op
en, FileAccess.Read, FileShare.ReadWrite, 1 << 20, true);
                using var dst = new FileStream(temp, FileMode.Cr
eate, FileAccess.Write, FileShare.None, 1 << 20, true);
                await src.CopyToAsync(dst);
            }

            long size = 0;
            try { size = new FileInfo(temp).Length; } catch { }

            var msg = new
            {
                type = "openFile",
                name,
                size,
                url = "https://pvspdf.file/" + Path.GetFileName(
temp),
                // Где документ лежит на диске: по этому пути ег
о откроют
                // из списка недавних. Временная копия удаляется
 при
                // следующем запуске, и ссылка на неё уже не раб
отает
                path = Path.GetFullPath(path)
            };
            PostToPage(msg);
        }
        catch (Exception ex)
        {
            AppLog.Error("открытие", "Не удалось открыть файл: "
```

```
 + path, ex);
          if (IsDisposed || Disposing) return;
          MessageBox.Show("Не удалось открыть файл:\r\n" + ex.
Message,
              "ПВ-Система PDF", MessageBoxButtons.OK, MessageB
oxIcon.Warning);
      }
  }

  // Последняя застава перед потерей работы: сюда приходит и к
рестик окна,
  // и Alt+F4, и закрытие через панель задач
  protected override void OnFormClosing(FormClosingEventArgs e
)
  {
      // Windows выключается или программа сама попросила закр
ыться —
      // задерживать нельзя
      bool forced = e.CloseReason == CloseReason.WindowsShutDo
wn
                  || e.CloseReason == CloseReason.TaskManagerCl
osing;

      if (!_closeConfirmed && !forced && _unsaved)
      {
          string what = string.IsNullOrWhiteSpace(_unsavedName
s)
              ? "В открытых документах есть изменения, которых
 нет в файлах."
              : "Не сохранены изменения в документах:\r\n\r\n"
 + _unsavedNames;

          var answer = MessageBox.Show(
              what + "\r\n\r\nЗакрыть программу и потерять эти
 изменения?",
              "ПВ-Система PDF",
              MessageBoxButtons.YesNo,
```

```
                MessageBoxIcon.Warning,
                MessageBoxDefaultButton.Button2);

            if (answer != DialogResult.Yes)
            {
                e.Cancel = true;
                base.OnFormClosing(e);
                return;
            }
        }

        base.OnFormClosing(e);
    }

    void OnWebMessage(object? sender, CoreWebView2WebMessageRece
ivedEventArgs e)
    {
        try
        {
            using var doc = JsonDocument.Parse(e.WebMessageAsJso
n);
            var root = doc.RootElement;
            string type = root.GetProperty("type").GetString() ?
? "";

            if (type == "setTitle")
            {
                string t = root.GetProperty("title").GetString()
 ?? "ПВ-Система PDF";
                Text = t;
            }
            else if (type == "unsaved")
            {
                // Интерфейс сообщает, есть ли работа, которой н
ет в файлах
                _unsaved = root.TryGetProperty("dirty", out var
d) && d.ValueKind == JsonValueKind.True;
```

```
                _unsavedNames = root.TryGetProperty("names", out
 var n)
                    ? (n.GetString() ?? "")
                    : "";
            }
            else if (type == "close")
            {
                // Закрытие изнутри программы: вопрос уже задан
там
                _closeConfirmed = true;
                Close();
            }
            else if (type == "minimize")
            {
                WindowState = FormWindowState.Minimized;
            }
            else if (type == "toggleMax")
            {
                WindowState = WindowState == FormWindowState.Max
imized
                    ? FormWindowState.Normal
                    : FormWindowState.Maximized;
            }
            else if (type == "print")
            {
                string b64 = root.GetProperty("data").GetString(
) ?? "";
                string fileName = root.TryGetProperty("name", ou
t var n)
                    ? (n.GetString() ?? "document.pdf")
                    : "document.pdf";
                string printer = root.TryGetProperty("printer",
out var pr)
                    ? (pr.GetString() ?? "")
                    : "";
                _ = PrintPdfAsync(b64, fileName, printer);
            }
```

```
            else if (type == "openPath")
            {
                // Повторное открытие из списка недавних докумен
тов
                string p = root.TryGetProperty("path", out var p
p) ? (pp.GetString() ?? "") : "";
                if (p.Length > 0 && File.Exists(p))
                {
                    _ = SendFileAsync(p);
                }
                else
                {
                    PostToPage(new
                    {
                        type = "openFileMissing",
                        path = p
                    });
                }
            }
            else if (type == "rememberPath")
            {
                // Документ выбрали в окне выбора файла или пере
тащили мышью:
                // сообщаем интерфейсу, где он лежит, чтобы запо
мнить
                // в недавних
                if (e.AdditionalObjects != null)
                {
                    foreach (var obj in e.AdditionalObjects)
                    {
                        if (obj is not CoreWebView2File f || str
ing.IsNullOrEmpty(f.Path)) continue;
                        long size = 0;
                        try { size = new FileInfo(f.Path).Length
; } catch { }
                        PostToPage(new
                        {
```

```
                            type = "filePath",
                            name = Path.GetFileName(f.Path),
                            size,
                            path = f.Path
                        });
                }
            }
        }
        else if (type == "printerSetup")
        {
            string p = root.TryGetProperty("printer", out va
r pn)
                ? (pn.GetString() ?? "")
                : "";
            OpenPrinterSetup(p);
        }
        else if (type == "saveFiles")
        {
            var items = new List<(string Name, string Data)>
();
            foreach (var it in root.GetProperty("files").Enu
merateArray())
            {
                items.Add((
                    it.GetProperty("name").GetString() ?? "f
ile",
                    it.GetProperty("data").GetString() ?? ""
));
            }
            _ = SaveManyAsync(items);
        }
        else if (type == "saveFile")
        {
            string b64 = root.GetProperty("data").GetString(
) ?? "";
            string fileName = root.TryGetProperty("name", ou
t var n)
```

```
                    ? (n.GetString() ?? "document.pdf")
                    : "document.pdf";
                _ = SaveFileAsync(b64, fileName);
            }
            else if (type == "installUpdate")
            {
                string url = root.TryGetProperty("url", out var
u) ? (u.GetString() ?? "") : "";
                string ver = root.TryGetProperty("version", out
var v) ? (v.GetString() ?? "") : "";
                _ = InstallUpdateAsync(url, ver);
            }
            else if (type == "cancelUpdate")
            {
                Updater.Stop();
            }
            else if (type == "listScanners")
            {
                _ = ListScannersAsync();
            }
            else if (type == "addScanner")
            {
                // Аппарат, который не отзывается на общий опрос
,
                // человек называет сам
                var item = new ManualScanners.Item
                {
                    Name = root.TryGetProperty("name", out var m
n) ? (mn.GetString() ?? "").Trim() : "",
                    Wia = root.TryGetProperty("wia", out var mw)
 && mw.GetBoolean(),
                    Feeder = root.TryGetProperty("feeder", out v
ar mf) && mf.GetBoolean(),
                    Duplex = root.TryGetProperty("duplex", out v
ar md) && md.GetBoolean(),
                };
```

```
                ManualScanners.Add(item);
                _ = ListScannersAsync();
            }
            else if (type == "chooseScanner")
            {
                // Окно выбора от самого драйвера. Выбранный апп
арат
                // сразу запоминаем — набирать название не нужно
                _ = ChooseScannerAsync();
            }
            else if (type == "removeScanner")
            {
                string name = root.TryGetProperty("name", out va
r rn) ? (rn.GetString() ?? "") : "";
                ManualScanners.Remove(name);
                _ = ListScannersAsync();
            }
            else if (type == "scanSelfTest")
            {
                _ = ScanSelfTestAsync();
            }
            else if (type == "openScanSetup")
            {
                string exe = root.TryGetProperty("path", out var
 sp) ? (sp.GetString() ?? "") : "";
                OpenScanSetup(exe);
            }
            else if (type == "scanCaps")
            {
                string dev = root.TryGetProperty("device", out v
ar sd) ? (sd.GetString() ?? "") : "";
                _ = ScanCapsAsync(dev);
            }
            else if (type == "scan")
            {
                var opt = new Scanner.Options
                {
```

```
                DeviceId = root.TryGetProperty("device", out
 var dv) ? (dv.GetString() ?? "") : "",
                Dpi = root.TryGetProperty("dpi", out var dp)
 ? dp.GetInt32() : 300,
                Color = root.TryGetProperty("color", out var
 cl) ? (cl.GetString() ?? "color") : "color",
                Feeder = root.TryGetProperty("feeder", out v
ar fd) && fd.GetBoolean(),
                Duplex = root.TryGetProperty("duplex", out v
ar dx) && dx.GetBoolean(),
                Limit = root.TryGetProperty("limit", out var
 lm) ? lm.GetInt32() : 0,
            };

            // Устройству от производителя нужно имя, а не с
истемный код
            if (opt.DeviceId.StartsWith("twain:"))
            {
                opt.Twain = true;
                opt.DeviceName = opt.DeviceId.Substring(6);
            }

            // Название нужно ВСЕГДА, не только устройствам
драйвера.
            // Запасной путь через драйвер производителя ище
т аппарат
            // по имени, а системный код ему ни о чём не гов
орит.
            // Без имени драйвер открывал первый попавшийся
сканер —
            // так вместо Kyocera появлялось окно EPSON
            if (string.IsNullOrWhiteSpace(opt.DeviceName))
            {
                opt.DeviceName = root.TryGetProperty("device
Name", out var dn)
                    ? (dn.GetString() ?? "")
                    : "";
```

```
                }

                _ = ScanAsync(opt);
            }
            else if (type == "cancelScan")
            {
                try { _scanCancel?.Cancel(); } catch { }
            }
            else if (type == "licenseSave")
            {
                // Ответ сервера принимаем только с верной подпи
сью
                string payload = root.TryGetProperty("payload",
out var pl) ? (pl.GetString() ?? "") : "";
                string sig = root.TryGetProperty("sig", out var
sg) ? (sg.GetString() ?? "") : "";
                bool ok = License.Save(payload, sig);
                SendUpdate(new
                {
                    type = "licenseState",
                    full = License.IsFull(),
                    accepted = ok,
                    org = License.Org,
                    key = License.Key,
                    until = License.Until,
                });
            }
            else if (type == "licenseClear")
            {
                License.Clear();
                SendUpdate(new { type = "licenseState", full = f
alse, accepted = true, org = "", key = "", until = "" });
            }
            else if (type == "scanDriverUi")
            {
                string dev = root.TryGetProperty("device", out v
ar sd) ? (sd.GetString() ?? "") : "";
```

```
                _ = DriverUiAsync(dev);
            }
            else if (type == "log")
            {
                // Ошибка из интерфейса программы — в тот же жур
нал
                string area = root.TryGetProperty("area", out va
r la) ? (la.GetString() ?? "интерфейс") : "интерфейс";
                string text = root.TryGetProperty("message", out
 var lm) ? (lm.GetString() ?? "") : "";
                string level = root.TryGetProperty("level", out
var ll) ? (ll.GetString() ?? "error") : "error";
                if (text.Length > 4000) text = text.Substring(0,
 4000) + "…";
                if (level == "warn") AppLog.Warn(area, text);
                else AppLog.Error(area, text);
            }
            else if (type == "openLogs")
            {
                AppLog.OpenFolder();
            }
        }
        catch (Exception ex)
        {
            AppLog.Error("команда", "Сбой при обработке команды
интерфейса", ex);
        }
    }

    // Обновление программы: скачиваем установщик, показывая ход
 загрузки,
    // затем закрываем программу и ставим новую версию. После ус
тановки
    // программа запускается снова — участие пользователя не тре
буется
    async Task InstallUpdateAsync(string url, string version)
    {
```

```
      if (string.IsNullOrWhiteSpace(url))
      {
          SendUpdate(new { type = "updateState", state = "erro
r", error = "Ссылка на обновление не получена." });
          return;
      }

      if (Updater.Busy) return;

      var cts = Updater.Begin();
      SendUpdate(new { type = "updateState", state = "start" }
);

      try
      {
          string setup = await Updater.DownloadAsync(url, vers
ion, (percent, got, total) =>
          {
              SendUpdate(new
              {
                  type = "updateState",
                  state = "progress",
                  percent,
                  loaded = got,
                  total,
              });
          }, cts.Token);

          SendUpdate(new { type = "updateState", state = "inst
alling" });
          await Task.Delay(700);

          Updater.RunInstaller(setup);

          // Помощник установки уже ждёт закрытия окна.
          // Человек согласился на обновление — лишний вопрос
не задаём
```

```
            await Task.Delay(400);
            _closeConfirmed = true;
            Close();
        }
        catch (OperationCanceledException)
        {
            SendUpdate(new { type = "updateState", state = "canc
elled" });
        }
        catch (Exception ex)
        {
            AppLog.Error("обновление", "Не удалось установить об
новление", ex);
            SendUpdate(new { type = "updateState", state = "erro
r", error = ex.Message });
        }
        finally
        {
            Updater.End();
        }
    }

    // Какое качество умеет аппарат. Спрашиваем в стороне от окн
а:
    // драйвер отвечает не мгновенно
    async Task ScanCapsAsync(string deviceId)
    {
        var dpi = await Task.Run(() => Scanner.Resolutions(devic
eId));
        SendUpdate(new { type = "scanCaps", device = deviceId, d
pi });
    }

    // Окно выбора сканера, нарисованное драйвером. Аппарат, мол
чащий
    // при обычном опросе, в нём обычно есть
    async Task ChooseScannerAsync()
```

```
    {
        var dev = await Task.Run(() => TwainBridge.Choose());

        if (dev == null)
        {
            SendUpdate(new
            {
                type = "scannerChosen",
                ok = false,
                steps = string.Join("\n", TwainBridge.ChooseLog)
,
            });
            return;
        }

        ManualScanners.Add(new ManualScanners.Item
        {
            Name = dev.Name,
            Wia = false,
            Feeder = dev.HasFeeder,
            Duplex = dev.HasDuplex,
        });

        SendUpdate(new { type = "scannerChosen", ok = true, name
 = dev.Name });
        await ListScannersAsync();
    }

    // Список сканеров опрашиваем в стороне от окна: у сетевых у
стройств
    // ответ занимает секунды, и программа не должна на это зами
рать
    async Task ListScannersAsync()
    {
        var list = await Task.Run(() => Scanner.List());
        string hint = list.Count > 0 ? "" : await Task.Run(() =>
 Scanner.Diagnose());
```

```
        // Программы настройки драйверов: у сетевых МФУ аппарат
        // добавляется в них, поэтому даём открыть их одной кноп
кой
        var setups = await Task.Run(() => Scanner.Setups());

        SendUpdate(new
        {
            type = "scanners",
            items = list.Select(d => new
            {
                id = d.Id,
                name = d.Name,
                feeder = d.HasFeeder,
                duplex = d.HasDuplex,
                twain = d.Twain,
                manual = d.Manual,
            }),
            hint,
            setups = setups.Select(s => new
            {
                name = s.Name,
                path = s.Path,
                main = s.Main,
                network = s.Network,
            }),

            // Имена, записанные в Windows. Подсказываем их при
ручном
            // добавлении: драйверу нужно точное название, а на
память
            // человек напишет «куесера» и аппарат не отзовётся
            known = await Task.Run(() => Scanner.Registered()),
        });
    }

    // Отчёт о поиске сканеров. Нужен, когда аппарат не находитс
```

```
  t(p.r)!).push(p);
  const rowsXml: string[] = [];
  for (let r = 0; r < row; r++) {
    const cells = (byRow.get(r) || [])
      .sort((a, b) => a.c - b.c)
      .map((p) => {
        const ref = `${colName(p.c)}${r + 1}`;
        if (p.num !== undefined) return `<c r="${ref}" s="${p.xf
  }"><v>${p.num}</v></c>`;
        if (p.body) return `<c r="${ref}" s="${p.xf}" t="inlineS
  tr"><is>${p.body}</is></c>`;
        return `<c r="${ref}" s="${p.xf}"/>`;
      })
      .join('');
    const h = heights.get(r);
    rowsXml.push(`<row r="${r + 1}"${h ? ` ht="${Math.round(h *
  100) / 100}" customHeight="1"` : ''}>${cells}</row>`);
  }

  // Поля и ориентация листа при печати — как у страницы докумен
  та
  const inch = (pt: number) => Math.round((pt / 72) * 1000) / 10
  00;
  const a4 = Math.abs(Math.min(pg.W, pg.H) - 595) < 8 && Math.ab
  s(Math.max(pg.W, pg.H) - 842) < 8;
  const page = `<pageMargins left="${inch(pg.margins.l)}" right=
  "${inch(pg.margins.r)}" top="${inch(pg.margins.t)}" bottom="${
  inch(pg.margins.b)}" header="0" footer="0"/><pageSetup${a4 ? '
   paperSize="9"' : ''} orientation="${pg.W > pg.H ? 'landscape'
   : 'portrait'}"/>`;

  // Сетку Excel прячем: лист выглядит как страница документа, а
   рамки
  // таблиц нарисованы свои
  return `<?xml version="1.0" encoding="UTF-8" standalone="yes"?
  >
<worksheet xmlns="http://schemas.openxmlformats.org/spreadsheetm
```

```
  l/2006/main"><sheetViews><sheetView workbookViewId="0" showGri
  dLines="0"/></sheetViews><sheetFormatPr defaultRowHeight="15"/
  >${cols}<sheetData>${rowsXml.join('')}</sheetData>${
    merges.length ? `<mergeCells count="${merges.length}">${merg
  es.map((m) => `<mergeCell ref="${m}"/>`).join('')}</mergeCells
  >` : ''
  }${page}</worksheet>`;
};

export const buildRichXlsx = (pages: RichPage[]) => {
  const st = new Styles();
  const list = pages.length ? pages : [];
  const sheets = list.map((pg) => sheetOf(pg, st));
  const names = list.map((_, i) => `Стр. ${i + 1}`);

  const workbook = `<?xml version="1.0" encoding="UTF-8" standal
  one="yes"?>
<workbook xmlns="http://schemas.openxmlformats.org/spreadsheetml
  /2006/main" xmlns:r="http://schemas.openxmlformats.org/officeD
  ocument/2006/relationships"><sheets>${names
    .map((n, i) => `<sheet name="${esc(n)}" sheetId="${i + 1}" r
  :id="rId${i + 1}"/>`)
    .join('')}</sheets></workbook>`;
  const wbRels = `<?xml version="1.0" encoding="UTF-8" standalon
  e="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/
  2006/relationships">${names
    .map(
      (_, i) =>
        `<Relationship Id="rId${i + 1}" Type="http://schemas.ope
  nxmlformats.org/officeDocument/2006/relationships/worksheet" T
  arget="worksheets/sheet${i + 1}.xml"/>`,
    )
    .join(
      '',
    )}<Relationship Id="rIdS" Type="http://schemas.openxmlformat
  s.org/officeDocument/2006/relationships/styles" Target="styles
```

```
  .xml"/></Relationships>`;
  const contentTypes = `<?xml version="1.0" encoding="UTF-8" sta
  ndalone="yes"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/con
  tent-types"><Default Extension="rels" ContentType="application
  /vnd.openxmlformats-package.relationships+xml"/><Default Exten
  sion="xml" ContentType="application/xml"/><Override PartName="
  /xl/workbook.xml" ContentType="application/vnd.openxmlformats-
  officedocument.spreadsheetml.sheet.main+xml"/><Override PartNa
  me="/xl/styles.xml" ContentType="application/vnd.openxmlformat
  s-officedocument.spreadsheetml.styles+xml"/>${names
    .map(
      (_, i) =>
        `<Override PartName="/xl/worksheets/sheet${i + 1}.xml" C
  ontentType="application/vnd.openxmlformats-officedocument.spre
  adsheetml.worksheet+xml"/>`,
    )
    .join('')}</Types>`;
  const rels = `<?xml version="1.0" encoding="UTF-8" standalone=
  "yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/
  2006/relationships"><Relationship Id="rId1" Type="http://schem
  as.openxmlformats.org/officeDocument/2006/relationships/office
  Document" Target="xl/workbook.xml"/></Relationships>`;

  return zipFiles([
    { name: '[Content_Types].xml', text: contentTypes },
    { name: '_rels/.rels', text: rels },
    { name: 'xl/workbook.xml', text: workbook },
    { name: 'xl/_rels/workbook.xml.rels', text: wbRels },
    { name: 'xl/styles.xml', text: st.xml() },
    ...sheets.map((text, i) => ({ name: `xl/worksheets/sheet${i
  + 1}.xml`, text })),
  ]);
};
```

```
// ===== src/lib/zip.ts =====
// Упаковка файлов в архив ZIP.
//
// И документ Word, и книга Excel внутри устроены одинаково: это
   архив
// с несколькими служебными файлами. Сборка архива тут общая, чт
  обы
// оба формата пользовались одним проверенным кодом.
//
// Файлы кладём без сжатия: так формат остаётся простым и надёжн
  ым.

const crcTable = (() => {
  const t = new Uint32Array(256);
  for (let n = 0; n < 256; n++) {
    let c = n;
    for (let k = 0; k < 8; k++) c = c & 1 ? 0xedb88320 ^ (c >>>
  1) : c >>> 1;
    t[n] = c >>> 0;
  }
  return t;
})();

const crc32 = (bytes: Uint8Array) => {
  let c = 0xffffffff;
  for (let i = 0; i < bytes.length; i++) c = crcTable[(c ^ bytes
  [i]) & 0xff] ^ (c >>> 8);
  return (c ^ 0xffffffff) >>> 0;
};

export type ZipEntry = { name: string; data: Uint8Array };

export const zip = (entries: ZipEntry[]) => {
  const chunks: Uint8Array[] = [];
  const central: Uint8Array[] = [];
  let offset = 0;
```

```
  const u32 = (v: number) => [v & 255, (v >>> 8) & 255, (v >>> 1
  6) & 255, (v >>> 24) & 255];
  const u16 = (v: number) => [v & 255, (v >>> 8) & 255];

  for (const e of entries) {
    const name = new TextEncoder().encode(e.name);
    const sum = crc32(e.data);

    const local = new Uint8Array([
      ...u32(0x04034b50),
      ...u16(20),
      ...u16(0),
      ...u16(0),
      ...u16(0),
      ...u16(0),
      ...u32(sum),
      ...u32(e.data.length),
      ...u32(e.data.length),
      ...u16(name.length),
      ...u16(0),
      ...name,
    ]);

    chunks.push(local, e.data);

    central.push(
      new Uint8Array([
        ...u32(0x02014b50),
        ...u16(20),
        ...u16(20),
        ...u16(0),
        ...u16(0),
        ...u16(0),
        ...u16(0),
        ...u32(sum),
        ...u32(e.data.length),
        ...u32(e.data.length),
```

```
        ...u16(name.length),
        ...u16(0),
        ...u16(0),
        ...u16(0),
        ...u16(0),
        ...u32(0),
        ...u32(offset),
        ...name,
      ]),
    );

    offset += local.length + e.data.length;
  }

  const dirSize = central.reduce((n, c) => n + c.length, 0);
  const end = new Uint8Array([
    ...u32(0x06054b50),
    ...u16(0),
    ...u16(0),
    ...u16(entries.length),
    ...u16(entries.length),
    ...u32(dirSize),
    ...u32(offset),
    ...u16(0),
  ]);

  const all = [...chunks, ...central, end];
  const total = all.reduce((n, c) => n + c.length, 0);
  const out = new Uint8Array(total);
  let at = 0;
  for (const c of all) {
    out.set(c, at);
    at += c.length;
  }
  return out;
};
```

```
// Короткая запись для файлов, которые задаются текстом
export const zipFiles = (files: { name: string; text: string }[]
  ) =>
  zip(files.map((f) => ({ name: f.name, data: new TextEncoder().
  encode(f.text) })));
```

```
// ===== src/pages/Admin.tsx =====
import { useCallback, useEffect, useState } from "react";
import Icon from "@/components/ui/icon";
import { LOGO_URL } from "@/lib/brand";
import AdminLogin from "@/components/admin/AdminLogin";
import LicenseForm from "@/components/admin/LicenseForm";
import HistoryPanel from "@/components/admin/HistoryPanel";
import MachinesPanel from "@/components/admin/MachinesPanel";
import ReleasesPanel from "@/components/admin/ReleasesPanel";
import BuildPanel from "@/components/admin/BuildPanel";
import TrialPanel from "@/components/admin/TrialPanel";
import SalesPanel from "@/components/admin/SalesPanel";
import PasswordForm from "@/components/admin/PasswordForm";
import {
  clearToken,
  deleteLicense,
  getToken,
  listLicenses,
  logout,
  type License,
  type Stats,
} from "@/lib/adminApi";
import { toast } from "@/hooks/use-toast";

const today = () => new Date().toISOString().slice(0, 10);

const statusOf = (l: License) => {
  if (l.status === "blocked")
    return {
      label: "Заблокирована",
      cls: "bg-destructive text-destructive-foreground",
    };
  if (l.valid_until < today())
    return { label: "Истекла", cls: "bg-foreground text-backgrou
  nd" };
  return { label: "Активна", cls: "bg-primary text-primary-foreg
  round" };
```

```
};

const Admin = () => {
  const [authed, setAuthed] = useState<boolean | null>(null);
  const [items, setItems] = useState<License[]>([]);
  const [stats, setStats] = useState<Stats>({
    total: 0,
    active: 0,
    expired: 0,
    blocked: 0,
  });
  const [search, setSearch] = useState("");
  const [loading, setLoading] = useState(false);
  const [form, setForm] = useState<{ open: boolean; item: Licens
  e | null }>({
    open: false,
    item: null,
  });
  const [tab, setTab] = useState<
    "licenses" | "sales" | "trial" | "history" | "releases" | "b
  uild"
  >("licenses");
  const [histFor, setHistFor] = useState<License | null>(null);
  const [passOpen, setPassOpen] = useState(false);
  const [machFor, setMachFor] = useState<License | null>(null);

  const load = useCallback(async (q = "") => {
    setLoading(true);
    try {
      const res = await listLicenses(q);
      setItems(res.items.filter((i) => i.status !== "deleted"));
      setStats(res.stats);
      return true;
    } catch {
      toast({ title: "Не удалось загрузить список" });
      return false;
    } finally {
```

```
    setLoading(false);
  }
}, []);

// Отдельная проверка входа не нужна: загрузка списка сама пок
ажет,
// действует ли вход. Это экономит один запрос при каждом откр
ытии
useEffect(() => {
  if (!getToken()) {
    setAuthed(false);
    return;
  }
  listLicenses("")
    .then((res) => {
      setItems(res.items.filter((i) => i.status !== "deleted")
);
      setStats(res.stats);
      setAuthed(true);
    })
    .catch(() => {
      clearToken();
      setAuthed(false);
    });
}, []);

const remove = async (l: License) => {
  if (!window.confirm(`Удалить лицензию «${l.org_name}»?`)) re
turn;
  await deleteLicense(l.id);
  setItems((list) => list.filter((x) => x.id !== l.id));
  toast({ title: "Лицензия удалена" });
};

const exit = async () => {
  await logout().catch(() => undefined);
  clearToken();
```

```
    setAuthed(false);
  };

  if (authed === null) {
    return (
      <div className="flex min-h-screen items-center justify-cen
  ter bg-muted">
        <Icon
          name="LoaderCircle"
          size={28}
          className="animate-spin text-primary"
        />
      </div>
    );
  }

  if (!authed) {
    return (
      <AdminLogin
        onDone={() => {
          setAuthed(true);
          load();
        }}
      />
    );
  }

  const cards = [
    { label: "Всего", value: stats.total, icon: "KeyRound" },
    { label: "Активных", value: stats.active, icon: "CircleCheck
  " },
    { label: "Истёкших", value: stats.expired, icon: "Clock" },
    { label: "Заблокировано", value: stats.blocked, icon: "Ban"
  },
  ];

  return (
```

```
  <div className="min-h-screen bg-background font-body text-fo
reground">
    <header className="border-b border-foreground bg-backgroun
d">
      <div className="mx-auto flex h-16 max-w-[1280px] items-c
enter gap-3 px-5">
        <img src={LOGO_URL} alt="" className="h-7 w-auto" />
        <span className="font-head text-[0.76rem] font-bold up
percase tracking-[0.12em]">
          Панель лицензий
        </span>
        <a
          href="/"
          className="ml-auto hidden items-center gap-2 border
border-border px-3 py-2 text-[0.78rem] transition-colors hover
:border-foreground sm:inline-flex"
        >
          <Icon name="ExternalLink" size={14} />К программе
        </a>
        <button
          onClick={() => setPassOpen(true)}
          title="Сменить пароль администратора"
          className="inline-flex items-center gap-2 border bor
der-border px-3 py-2 text-[0.78rem] transition-colors hover:bo
rder-foreground"
        >
          <Icon name="KeyRound" size={14} />
          <span className="hidden sm:inline">Пароль</span>
        </button>
        <button
          onClick={exit}
          className="inline-flex items-center gap-2 border bor
der-foreground px-3 py-2 font-head text-[0.7rem] font-bold upp
ercase tracking-[0.1em] transition-colors hover:bg-foreground
hover:text-background"
        >
          <Icon name="LogOut" size={14} />
```

```
          Выйти
        </button>
      </div>
    </header>

    <main className="mx-auto max-w-[1280px] px-5 py-8">
      <div className="grid grid-cols-2 border-l border-t borde
r-border lg:grid-cols-4">
        {cards.map((c) => (
          <div key={c.label} className="border-b border-r bord
er-border p-5">
            <Icon name={c.icon} size={18} className="text-prim
ary" />
            <div className="mt-3 font-head text-[1.8rem] font-
black leading-none">
              {c.value}
            </div>
            <div className="mt-1 text-[0.78rem] uppercase trac
king-[0.1em] text-muted-foreground">
              {c.label}
            </div>
          </div>
        ))}
      </div>

      <div className="mt-8 flex border border-border">
        {(
          [
            ["licenses", "Лицензии"],
            ["sales", "Продажи"],
            ["trial", "Пробный режим"],
            ["history", "История проверок"],
            ["releases", "Версии программы"],
            ["build", "Сборка"],
          ] as const
        ).map(([id, label]) => (
          <button
```

```
            key={id}
            onClick={() => setTab(id)}
            className={`flex-1 py-3 font-head text-[0.74rem] f
ont-bold uppercase tracking-[0.08em] transition-colors ${
              tab === id ? "bg-foreground text-background" : "
hover:bg-card"
            }`}
          >
            {label}
          </button>
        ))}
      </div>

      {tab === "sales" ? (
        <SalesPanel />
      ) : tab === "trial" ? (
        <TrialPanel />
      ) : tab === "build" ? (
        <BuildPanel />
      ) : tab === "releases" ? (
        <ReleasesPanel />
      ) : tab === "history" ? (
        <div className="mt-6">
          <HistoryPanel />
        </div>
      ) : (
        <>
          <div className="mt-6 flex flex-wrap items-center gap
-3">
            <div className="flex flex-1 items-center border bo
rder-border bg-background">
              <input
                value={search}
                onChange={(e) => setSearch(e.target.value)}
                onKeyDown={(e) => e.key === "Enter" && load(se
arch)}
                placeholder="Поиск по организации или ключу"
```

```
                <input
                  value={search}
                  onChange={(e) => setSearch(e.target.value)}
                  onKeyDown={(e) => e.key === "Enter" && load(search)}
                  placeholder="Поиск по организации или ключу…"
                  className="w-full bg-transparent px-3 py-3 tex
t-[0.9rem] outline-none"
                />
                <button
                  onClick={() => load(search)}
                  className="px-3 py-3 text-primary hover:bg-car
d"
                >
                  <Icon name="Search" size={16} />
                </button>
              </div>
              <button
                onClick={() => setForm({ open: true, item: null
})}
                className="btn-block"
              >
                <Icon name="Plus" size={16} />
                Новая лицензия
              </button>
            </div>

            <div className="mt-6 overflow-x-auto border border-b
order">
              <table className="w-full min-w-[820px] border-coll
apse text-left">
                <thead>
                  <tr>
                    {[
                      "Организация",
                      "Ключ активации",
                      "Действует до",
                      "Мест",
                      "Проверок",
                      "Статус",
                      "",
                    ].map((h) => (
```

```
                    <th
                      key={h}
                      className="px-4 py-3 font-head text-[0.7
rem] font-bold uppercase tracking-[0.1em] text-muted-foregroun
d"
                    >
                      {h}
                    </th>
                  ))}
                </tr>
              </thead>
              <tbody>
                {loading && (
                  <tr>
                    <td
                      colSpan={7}
                      className="px-4 py-10 text-center text-m
uted-foreground"
                    >
                      <Icon
                        name="LoaderCircle"
                        size={20}
                        className="mx-auto animate-spin text-p
rimary"
                      />
                    </td>
                  </tr>
                )}
                {!loading && !items.length && (
                  <tr>
                    <td
                      colSpan={7}
                      className="px-4 py-12 text-center text-m
uted-foreground"
                    >
                      Лицензий пока нет — создайте первую
                    </td>
```

```
                  </tr>
                )}
                {!loading &&
                  items.map((l) => {
                    const s = statusOf(l);
                    return (
                      <tr
                        key={l.id}
                        className="border-b border-border tran
sition-colors last:border-0 hover:bg-card"
                      >
                        <td className="px-4 py-3">
                          <div className="font-head text-[0.9r
em] font-bold">
                            {l.org_name}
                          </div>
                          {l.contact && (
                            <div className="text-[0.78rem] tex
t-muted-foreground">
                              {l.contact}
                            </div>
                          )}
                        </td>
                        <td className="px-4 py-3">
                          <button
                            onClick={() => {
                              navigator.clipboard?.writeText(l
.license_key);
                              toast({ title: "Ключ скопирован"
 });
                            }}
                            className="group flex items-center
 gap-2 font-head text-[0.82rem] font-bold tracking-[0.05em]"
                            title="Скопировать"
                          >
                            {l.license_key}
                            <Icon
```

```
                                name="Copy"
                                size={13}
                                className="opacity-0 group-hover
:opacity-60"
                              />
                            </button>
                          </td>
                          <td className="px-4 py-3 text-[0.86rem
]">
                            {l.valid_until}
                          </td>
                          <td className="px-4 py-3 text-[0.86rem
]">
                            {l.seats}
                          </td>
                          <td className="px-4 py-3 text-[0.86rem
] text-muted-foreground">
                            {l.activations}
                          </td>
                          <td className="px-4 py-3">
                            <span
                              className={`inline-block px-2 py-1
 font-head text-[0.68rem] font-bold uppercase tracking-[0.08em
] ${s.cls}`}
                            >
                              {s.label}
                            </span>
                          </td>
                          <td className="px-4 py-3">
                            <div className="flex justify-end gap
-1">
                              <button
                                onClick={() => setMachFor(l)}
                                className="p-2 hover:text-primar
y"
                                title="Компьютеры лицензии"
                              >
```

```
                                <Icon name="MonitorSmartphone" s
ize={15} />
                              </button>
                              <button
                                onClick={() => setHistFor(l)}
                                className="p-2 hover:text-primar
y"
                                title="История проверок"
                              >
                                <Icon name="History" size={15} /
>
                              </button>
                              <button
                                onClick={() => setForm({ open: t
rue, item: l })}
                                className="p-2 hover:text-primar
y"
                                title="Изменить"
                              >
                                <Icon name="Pencil" size={15} />
                              </button>
                              <button
                                onClick={() => remove(l)}
                                className="p-2 hover:text-destru
ctive"
                                title="Удалить"
                              >
                                <Icon name="Trash2" size={15} />
                              </button>
                            </div>
                          </td>
                        </tr>
                      );
                    })}
                  </tbody>
                </table>
              </div>
```

```
          </>
        )}
      </main>

      {machFor && (
        <MachinesPanel
          licenseId={machFor.id}
          title={machFor.org_name}
          seats={machFor.seats}
          onClose={() => setMachFor(null)}
        />
      )}

      {histFor && (
        <HistoryPanel
          licenseId={histFor.id}
          title={`История: ${histFor.org_name}`}
          onClose={() => setHistFor(null)}
        />
      )}

      {form.open && (
        <LicenseForm
          item={form.item}
          onClose={() => setForm({ open: false, item: null })}
          onSaved={() => {
            setForm({ open: false, item: null });
            load(search);
          }}
        />
      )}

      {passOpen && <PasswordForm onClose={() => setPassOpen(fals
  e)} />}
    </div>
  );
};
```

```
export default Admin;
```

```
// ===== src/pages/NotFound.tsx =====
import { useLocation } from "react-router-dom";
import { useEffect } from "react";

const NotFound = () => {
  const location = useLocation();

  useEffect(() => {
    console.error(
      "404 Error: User attempted to access non-existent route:",
      location.pathname
    );
  }, [location.pathname]);

  return (
    <div className="min-h-screen flex items-center justify-cente
  r bg-gray-100">
      <div className="text-center">
        <h1 classNamе="text-4xl font-bold mb-4">404</h1>
        <p className="text-xl text-gray-600 mb-4">Упс! Страница
  не найдена</p>
        <a href="/" className="text-blue-500 hover:text-blue-700
   underline">
          Вернуться на главную
        </a>
      </div>
    </div>
  );
};

export default NotFound;
```

```
// ===== src/pages/Thanks.tsx =====
import { useCallback, useEffect, useRef, useState } from 'react'
  ;
import { useSearchParams } from 'react-router-dom';
import Icon from '@/components/ui/icon';
import { APP_NAME, LOGO_URL } from '@/lib/brand';
import { orderStatus } from '@/lib/adminApi';
import { toast } from '@/hooks/use-toast';

// Банк присылает клиента сюда сразу после оплаты, но подтвержде
  ние платежа
// приходит отдельным путём и может немного отстать. Поэтому стр
  аница
// сама переспрашивает о заказе, а не пугает человека пустотой
const STEP_MS = 2500;
const GIVE_UP_MS = 3 * 60 * 1000;

type State = 'waiting' | 'paid' | 'slow' | 'unknown';

const Thanks = () => {
  const [params] = useSearchParams();
  const token = params.get('tok') || params.get('Shp_tok') || ''
  ;

  const [state, setState] = useState<State>(token ? 'waiting' :
  'unknown');
  const [key, setKey] = useState('');
  const [title, setTitle] = useState('');
  const timer = useRef<number | null>(null);

  const stop = useCallback(() => {
    if (timer.current) {
      window.clearInterval(timer.current);
      timer.current = null;
    }
  }, []);
```

```
  useEffect(() => {
    if (!token) return;
    const started = Date.now();

    const ask = async () => {
      try {
        const r = await orderStatus(token);
        if (r.title) setTitle(r.title);

        if (r.paid && r.license_key) {
          stop();
          setKey(r.license_key);
          setState('paid');
          return;
        }
        // Банк подтверждает оплату не мгновенно — ждём, но не б
  есконечно
        if (Date.now() - started > GIVE_UP_MS) {
          stop();
          setState('slow');
        }
      } catch {
        stop();
        setState('unknown');
      }
    };

    void ask();
    timer.current = window.setInterval(ask, STEP_MS);
    return stop;
  }, [token, stop]);

  const copy = () => {
    navigator.clipboard?.writeText(key);
    toast({ title: 'Ключ скопирован' });
  };
```

```
return (
  <div className="flex min-h-screen flex-col bg-background">
    <header className="border-b border-border">
      <div className="mx-auto flex max-w-[880px] items-center
gap-2.5 px-6 py-4">
        <img src={LOGO_URL} alt="" className="h-7 w-auto" />
        <span className="font-head text-[0.8rem] font-bold upp
ercase tracking-[0.12em]">
          {APP_NAME}
        </span>
      </div>
    </header>

    <main className="mx-auto w-full max-w-[560px] flex-1 px-6
py-14">
      {state === 'paid' && (
        <>
          <div className="flex items-center gap-4">
            <span className="flex h-14 w-14 shrink-0 items-cen
ter justify-center border border-primary bg-primary/5">
              <Icon name="ShieldCheck" size={28} className="te
xt-primary" />
            </span>
            <div>
              <h1 className="font-head text-[1.5rem] font-blac
k uppercase leading-none">
                Спасибо за покупку
              </h1>
              <p className="mt-1.5 text-[0.9rem] text-muted-fo
reground">
                Оплата прошла. {title && `${title}.`}
              </p>
            </div>
          </div>

          <div className="mt-8">
            <div className="label-caps">Ваш ключ активации</di
```

```
v>
            <div className="mt-2 flex items-stretch border bor
der-foreground">
              <code className="flex-1 bg-card px-4 py-4 text-c
enter font-head text-[1.05rem] font-bold tracking-[0.08em]">
                {key}
              </code>
              <button
                onClick={copy}
                title="Скопировать"
                className="border-l border-foreground px-5 tra
nsition-colors hover:bg-card"
              >
                <Icon name="Copy" size={16} />
              </button>
            </div>
          </div>

          <div className="mt-8 border-l-2 border-foreground pl
-4">
            <div className="label-caps">Что дальше</div>
            <ol className="mt-3 space-y-2.5 text-[0.9rem]">
              <li className="flex gap-2.5">
                <span className="font-head font-bold text-mute
d-foreground">1.</span>
                Вернитесь в программу — если она открыта, полн
ая версия
                включится сама, вводить ключ не нужно
              </li>
              <li className="flex gap-2.5">
                <span className="font-head font-bold text-mute
d-foreground">2.</span>
                Если программа была закрыта, откройте её, нажм
ите
                «Активировать полную версию» и введите ключ
              </li>
            </ol>
```

```
          </div>

          <div className="mt-8 flex items-start gap-2.5 border
 border-border bg-card px-4 py-3.5 text-[0.85rem]">
            <Icon name="Mail" size={15} className="mt-0.5 shri
nk-0 text-muted-foreground" />
            <span>
              Копия ключа отправлена вам на почту. Сохраните п
исьмо: ключ
              понадобится при переустановке программы.
            </span>
          </div>
        </>
      )}

      {state === 'waiting' && (
        <div className="py-10 text-center">
          <Icon name="LoaderCircle" size={34} className="mx-au
to animate-spin text-primary" />
          <h1 className="mt-6 font-head text-[1.3rem] font-bla
ck uppercase">
            Подтверждаем оплату
          </h1>
          <p className="mx-auto mt-3 max-w-[380px] text-[0.9re
m] leading-relaxed text-muted-foreground">
            Это занимает несколько секунд. Не закрывайте стран
ицу — ключ
            появится здесь сам.
          </p>
        </div>
      )}

      {state === 'slow' && (
        <div className="py-10 text-center">
          <Icon name="Clock" size={30} className="mx-auto text
-muted-foreground" />
          <h1 className="mt-6 font-head text-[1.3rem] font-bla
```

```
ck uppercase">
            Оплата ещё обрабатывается
          </h1>
          <p className="mx-auto mt-3 max-w-[420px] text-[0.9re
m] leading-relaxed text-muted-foreground">
            Иногда банк подтверждает платёж дольше обычного. К
люч придёт
            к вам на почту, как только оплата пройдёт — деньги
 не потеряются.
          </p>
          <button
            onClick={() => window.location.reload()}
            className="btn-block mx-auto mt-6"
          >
            <Icon name="RefreshCw" size={15} />
            Проверить ещё раз
          </button>
        </div>
      )}

      {state === 'unknown' && (
        <div className="py-10 text-center">
          <Icon name="CircleHelp" size={30} className="mx-auto
 text-muted-foreground" />
          <h1 className="mt-6 font-head text-[1.3rem] font-bla
ck uppercase">
            Не нашли заказ
          </h1>
          <p className="mx-auto mt-3 max-w-[420px] text-[0.9re
m] leading-relaxed text-muted-foreground">
            Похоже, ссылка открыта без сведений о покупке. Есл
и вы оплатили —
            ключ придёт на почту, указанную при оформлении.
          </p>
        </div>
      )}
    </main>
```

```
      <footer className="border-t border-border">
        <div className="mx-auto max-w-[880px] px-6 py-4 text-[0.
  78rem] text-muted-foreground">
          Возникли вопросы по оплате — ответьте на письмо с ключ
  ом, поможем.
        </div>
      </footer>
    </div>
  );
};

export default Thanks;
```

```
// ===== src/types/media.d.ts =====
// Описания для библиотек чтения снимков: своих типов они не пос
  тавляют

declare module 'utif' {
  export type IFD = {
    width: number;
    height: number;
    [key: string]: unknown;
  };
  const UTIF: {
    decode: (buffer: ArrayBuffer | Uint8Array) => IFD[];
    decodeImage: (buffer: ArrayBuffer | Uint8Array, ifd: IFD) =>
   void;
    toRGBA8: (ifd: IFD) => Uint8Array;
  };
  export default UTIF;
}

declare module 'heic-decode' {
  const decode: (input: { buffer: Uint8Array }) => Promise<{
    width: number;
    height: number;
    data: Uint8Array;
  }>;
  export default decode;
}
```